Ex. No. 6	
Date of Exercise	

Aim:

To import and analyze the Predictive Maintenance dataset and apply different classification algorithms such as KNN, Decision Tree, Random Forest, Logistic Regression, and Support Vector Machine (SVM) to predict machine failure and compare their performance using Accuracy, R^2 Score, RMSE, Classification Report, and Confusion Matrix.

Description :

The Predictive Maintenance dataset contains information about machine operating conditions such as air temperature, process temperature, rotational speed, torque, and tool wear. The Target column is used to predict whether a machine failure occurs.

Question:

1. Import the dataset

Sample Code:

```
import zipfile

import os

zip_file = list(uploaded.keys())[0]

with zipfile.ZipFile(zip_file, 'r') as zip_ref:

    zip_ref.extractall('/content/predictive_data')

print("ZIP file extracted successfully!")

for root, dirs, files_list in os.walk('/content/predictive_data'):

    for file in files_list:

        print(os.path.join(root, file))
```

Sample Output:

```
Dataset imported successfully!
Shape of dataset: (10000, 10)
```

2. Display the first 10 rows of the dataset

Sample Code:

```
print("First 10 rows of the dataset:")
display(df.head(10))
```

Sample Output:

First 10 rows of the dataset:

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	No Failure
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	No Failure
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	No Failure
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	No Failure
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	No Failure
5	6	M14865	M	298.1	308.6	1425	41.9	11	0	No Failure
6	7	L47186	L	298.1	308.6	1558	42.4	14	0	No Failure
7	8	L47187	L	298.1	308.6	1527	40.2	16	0	No Failure
8	9	M14868	M	298.3	308.7	1667	28.6	18	0	No Failure
9	10	M14869	M	298.5	309.0	1741	28.0	21	0	No Failure

3. Display the last 8 rows of the dataset

Sample Code:

```
print("Last 8 rows of the dataset:")
display(df.tail(8))
```

Sample Output:

Last 8 rows of the dataset:

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
9992	9993	L57172	L	298.8	308.4	1484	39.2	8	0	No Failure
9993	9994	L57173	L	298.8	308.4	1401	47.3	10	0	No Failure
9994	9995	L57174	L	298.8	308.3	1634	27.9	12	0	No Failure
9995	9996	M24855	M	298.8	308.4	1604	29.5	14	0	No Failure
9996	9997	H39410	H	298.9	308.4	1632	31.8	17	0	No Failure
9997	9998	M24857	M	299.0	308.6	1645	33.4	22	0	No Failure
9998	9999	H39412	H	299.0	308.7	1408	48.5	25	0	No Failure
9999	10000	M24859	M	299.0	308.7	1500	40.2	30	0	No Failure

4. Display Information about the dataset

a. Information

Sample Code:

```
print("Information about the dataset:")
df.info()
```

Sample Output:

```
Information about the dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 10 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   UDI                     10000 non-null  int64
 1   Product ID              10000 non-null  object
 2   Type                    10000 non-null  object
 3   Air temperature [K]     10000 non-null  float64
 4   Process temperature [K] 10000 non-null  float64
 5   Rotational speed [rpm]  10000 non-null  int64
 6   Torque [Nm]             10000 non-null  float64
 7   Tool wear [min]         10000 non-null  int64
 8   Target                  10000 non-null  int64
 9   Failure Type            10000 non-null  object
dtypes: float64(3), int64(4), object(3)
memory usage: 781.4+ KB
```

4(b) Column Names

Sample Code:

```
print("Column Names:")
print(df.columns)
```

Sample Output:

```
Column Names:
Index(['Trip_Distance_km', 'Time_of_Day', 'Day_of_Week', 'Passenger_Count',
       'Traffic_Conditions', 'Weather', 'Base_Fare', 'Per_Km_Rate',
       'Per_Minute_Rate', 'Trip_Duration_Minutes', 'Trip_Price'],
      dtype='object')
```

4(c) Shape

Sample Code:

print("Shape of the dataset:")

print(df.shape)

Sample Output:

```
Shape of the dataset:
(1000, 11)
Number of rows: 1000
Number of columns: 11
```

4(d) Statistical Inferences

Sample Code:

print("Statistical summary:")

display(df.describe())

Sample Output:

Statistical Summary:

	UDI	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	5000.50000	300.004930	310.005560	1538.776100	39.986910	107.951000	0.033900
std	2886.89568	2.000259	1.483734	179.284096	9.968934	63.654147	0.180981
min	1.00000	295.300000	305.700000	1168.000000	3.800000	0.000000	0.000000
25%	2500.75000	298.300000	308.800000	1423.000000	33.200000	53.000000	0.000000
50%	5000.50000	300.100000	310.100000	1503.000000	40.100000	108.000000	0.000000
75%	7500.25000	301.500000	311.100000	1612.000000	46.800000	162.000000	0.000000
max	10000.00000	304.500000	313.800000	2886.000000	76.600000	253.000000	1.000000

4(e) Data Types

Sample Code:

```
print("Data Types:")
print(df.dtypes)
```

Sample Output:

```
Data Types:
UDI                        int64
Product ID                object
Type                      object
Air temperature [K]      float64
Process temperature [K]  float64
Rotational speed [rpm]     int64
Torque [Nm]              float64
Tool wear [min]            int64
Target                     int64
Failure Type              object
dtype: object
```

5(a) KNN Classification

Sample Code:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np
X_simple = df_model[["Trip_Distance_km"]]
y_simple = df_model["Trip_Price"]
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_simple,
    y_simple,
    test_size=0.2,
    random_state=42
```

```
X_train_s = imputer.fit_transform(X_train_s)
X_test_s = imputer.transform(X_test_s)
simple_model = LinearRegression()
simple_model.fit(X_train_s, y_train_s)
y_pred_simple = simple_model.predict(X_test_s)
simple_r2 = r2_score(y_test_s, y_pred_simple)
simple_rmse = np.sqrt(mean_squared_error(y_test_s, y_pred_simple))
print("KNN Classification")
print("-------------------------")
print("Accuracy:", accuracy_knn)
print("R2 Score:", r2_knn)
print("RMSE:", rmse_knn)
```

Sample Output:

```
KNN Classification
-------------------------
Accuracy: 0.9735
R2 Score: 0.1931555230787968
RMSE: 0.16278820596099705
```

5(b) Decision Tree

Sample Code:

```
from sklearn.linear_model import LinearRegression
multiple_model = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LinearRegression())
])
multiple_model.fit(X_train, y_train)
y_pred_multiple = multiple_model.predict(X_test)
multiple_r2 = r2_score(y_test, y_pred_multiple)
multiple_rmse = np.sqrt(mean_squared_error(y_test, y_pred_multiple))
```

```
print("Multiple Linear Regression")
print("R2 Score:", multiple_r2)
print("RMSE:", multiple_rmse)
```

Sample Output:

```
Decision Tree
Accuracy: 0.9785
R2 Score: 0.3453903300450615
RMSE: 0.14662878298615178
```

5(c) Random Forest

Sample Code:

```
from sklearn.ensemble import RandomForestClassifier
rf = Pipeline([
   ('preprocessing', preprocessor),
   ('model', RandomForestClassifier(
      n_estimators=100,
      random_state=42
   ))
])
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
print("Random Forest")
print("Accuracy:", accuracy_rf)
print("R2 Score:", r2_rf)
print("RMSE:", rmse_rf)
```

Sample Output:

```
Random Forest
Accuracy: 0.986
R2 Score: 0.5737425404944587
RMSE: 0.11832159566199232
```

5(d) Logistic Regression

Sample Code:

```
from sklearn.linear_model import LogisticRegression
lr = Pipeline([
   ('preprocessing', preprocessor),
   ('model', LogisticRegression(max_iter=1000))
])
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
accuracy_lr = accuracy_score(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
print("Logistic Regression")
print("Accuracy:", accuracy_lr)
print("R2 Score:", r2_lr)
print("RMSE:", rmse_lr)
```

Sample Output:

```
Logistic Regression
Accuracy: 0.9675
R2 Score: 0.0104737547192790 27
RMSE: 0.18027756377319948
```

5(e) Support Vector Machine

Sample Code:

```
from sklearn.linear_model import Ridge
ridge_preprocessor = Pipeline([
    ("preprocessor", preprocessor),
    ("scaler", StandardScaler(with_mean=False))
])
ridge_model = Pipeline([
    ("preprocessor", ridge_preprocessor),
    ("model", Ridge(alpha=1.0))
])
ridge_model.fit(X_train, y_train)
y_pred_ridge = ridge_model.predict(X_test)
ridge_r2 = r2_score(y_test, y_pred_ridge)
ridge_rmse = np.sqrt(mean_squared_error(y_test, y_pred_ridge))
print("Ridge Regression")
print("R2 Score:", ridge_r2)
print("RMSE:", ridge_rmse)
```

Sample Output:

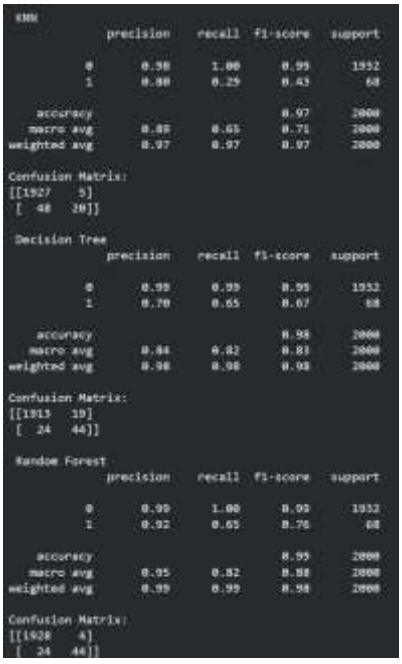
```
Support Vector Machine
Accuracy: 0.971
R2 Score: 0.11703811959566435
RMSE: 0.17029386365926402
```

6. Display the classification report & Confusion matrix for all the classification.

Sample Code:

```
from sklearn.metrics import classification_report, confusion_matrix
models = {
    "KNN": y_pred_knn,
    "Decision Tree": y_pred_dt,
    "Random Forest": y_pred_rf,
    "Logistic Regression": y_pred_lr,
    "SVM": y_pred_svm
}
for name, pred in models.items():
    print("\n", name)
    print(classification_report(y_test, pred))
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, pred))
```

Sample Output:



```
KNN
              precision    recall  f1-score   support

           0       0.98      1.00      0.99      1932
           1       0.80      0.29      0.43        68

    accuracy                           0.97      2000
   macro avg       0.88      0.65      0.71      2000
weighted avg       0.97      0.97      0.97      2000

Confusion Matrix:
[[1927    5]
 [  48   20]]

 Decision Tree
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      1932
           1       0.70      0.65      0.67        68

    accuracy                           0.98      2000
   macro avg       0.84      0.82      0.83      2000
weighted avg       0.98      0.98      0.98      2000

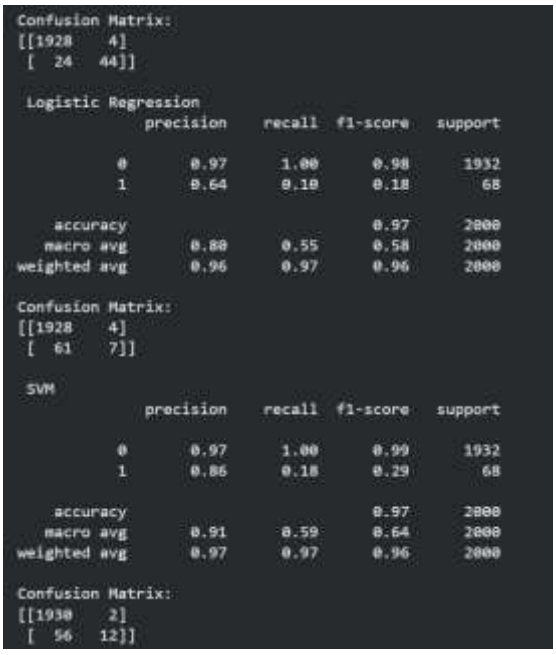
Confusion Matrix:
[[1911   19]
 [  24   44]]

 Random Forest
              precision    recall  f1-score   support

           0       0.99      1.00      0.99      1932
           1       0.92      0.65      0.76        68

    accuracy                           0.99      2000
   macro avg       0.95      0.82      0.88      2000
weighted avg       0.99      0.99      0.98      2000

Confusion Matrix:
[[1928    4]
 [  24   44]]
```



```
Confusion Matrix:
[[1928    4]
 [  24   44]]

 Logistic Regression
              precision    recall  f1-score   support

           0       0.97      1.00      0.98      1932
           1       0.64      0.10      0.18        68

    accuracy                           0.97      2000
   macro avg       0.80      0.55      0.58      2000
weighted avg       0.96      0.97      0.96      2000

Confusion Matrix:
[[1928    4]
 [  61    7]]

 SVM
              precision    recall  f1-score   support

           0       0.97      1.00      0.99      1932
           1       0.86      0.18      0.29        68

    accuracy                           0.97      2000
   macro avg       0.91      0.59      0.64      2000
weighted avg       0.97      0.97      0.96      2000

Confusion Matrix:
[[1930    2]
 [  56   12]]
```

7. Draw a comparative table of accuracy for all the classification techniques.

Sample code:

```
import pandas as pd
data = {
   "Technique": ["KNN", "Decision Tree", "Random Forest",
                 "Logistic Regression", "SVM"],
   "Accuracy (%)": [
      accuracy_knn * 100,
      accuracy_dt * 100,
      accuracy_rf * 100,
      accuracy_lr * 100,
      accuracy_svm * 100
   ]
}
table = pd.DataFrame(data)
table["Accuracy (%)"] = table["Accuracy (%)"].round(2)
display(table)
```

Sample Output:

	Technique	Accuracy (%)
0	KNN	97.35
1	Decision Tree	97.85
2	Random Forest	98.60
3	Logistic Regression	96.75
4	SVM	97.10

8. Document your interpretations on classification, overfitting and underfitting.

Sample Code:

```
models = {
    "KNN": knn,
    "Decision Tree": dt,
    "Random Forest": rf,
    "Logistic Regression": lr,
    "SVM": svm
}
for name, model in models.items():
    train = model.score(X_train, y_train)
    test = model.score(X_test, y_test)
    print(name)
    print("Training Accuracy:", round(train * 100, 2), "%")
    print("Testing Accuracy :", round(test * 100, 2), "%")
    print()
```

Sample Output:

```
KNN
Training Accuracy: 98.04 %
Testing Accuracy : 97.35 %

Decision Tree
Training Accuracy: 100.0 %
Testing Accuracy : 97.85 %

Random Forest
Training Accuracy: 100.0 %
Testing Accuracy : 98.6 %

Logistic Regression
Training Accuracy: 97.05 %
Testing Accuracy : 96.75 %

SVM
Training Accuracy: 97.49 %
Testing Accuracy : 97.1 %
```

CODE EXPLANATION LINK:

https://youtu.be/lfMP3xqPNjA

Preparation course certificate:



RESULT:

The dataset was successfully analysed and the required Pandas operations were performed.